



Amlogic

Model Conversion User Guide

Revision: 1.6

Release Date: 2024-07-05

Copyright

© 2024 Amlogic. All rights reserved. No part of this document may be reproduced, transmitted, transcribed, or translated into any language in any form or by any means without the written permission of Amlogic.

Trademarks

 , and other Amlogic icons are trademarks of Amlogic companies. All other trademarks and registered trademarks are property of their respective holders.

Disclaimer

Amlogic may make improvements and/or changes in this document or in the product described in this document at any time and without notice.

This product is not intended for use in medical, life saving, or life sustaining applications, nor in applications where failure or malfunction of an Amlogic product can reasonably be expected to result in personal injury, death or severe property or environmental damage.

Circuit diagrams and other information relating to products of Amlogic are included as a means of illustrating typical applications. Consequently, complete information sufficient for production design is not necessarily given. Though every effort has been made to ensure that this document is current and accurate, Amlogic makes no representations or warranties with respect to the accuracy or completeness of the contents presented in this document.

Amlogic products may be subject to unidentified vulnerabilities or may support established security standards or specifications with known limitations. Amlogic accepts no liability for any vulnerability.

Use of the materials may require a license of intellectual property from a third party or from Amlogic. This document conveys no express or implied licenses to any intellectual property rights belonging to Amlogic or any other party. Any links to third-party websites included in this document are for your convenience only. Amlogic does not endorse and is not responsible for such websites and their practices, including privacy practices, availability, and content.

Amlogic shall in no event be liable for any claims relating to or arising out of this document or any use or inability to use hereof.

Contact Information

- Website: www.amlogic.com
- Pre-sales consultation: contact@amlogic.com
- Technical support: support@amlogic.com

Revision History

Issue 1.6 (2024-07-05)

This is the seventh official release

- 1、 Updated the Framework version information supported by adla_tool in chapter 2.2.3
- 2、 Add new Platform information in chapter 3.4.1
- 3、 Updated Quantize optimization tool in chapter 4.2
- 4、 Added new chapter 4.4 NNIDE_Tool
- 5、 Remove Chapter 5
- 6、 Added method to obtain tools in Chapter 2.2.2

Issue 1.5 (2023-10-20)

This is the sixth official release.

1. Update the Framework version information supported by adla_tool
2. Added feature quantification algorithm and corresponding parameter description:
 - (1)hist_percentile
 - (2)Hist_asymm_percentile
 - (3)history_moving-avg
3. Added introductions to other tools, including:
 - (1)adla_info_tool
 - (2)quantize parameter search tool
 - (3)NNAPI_Optimize_tool

Issue 1.4 (2023-5-29)

This is the fifth official release.

1. Added support for uint8 quantization conversion.
2. Added support for exporting demo code.
3. Added integrated adla_info tool.
4. Added support for specifying the output layer of ONNX and Tflite models.

Issue 1.3 (2023-3-20)

This is the fourth official release.

1. New feature quantization algorithm
 - (1) kl_symm_divergence
 - (2) kl_asymm_divergence
 - (3) moving-avg、percentile
2. Added Feature Quantization Algorithm Parameters
 - (1) --quantize-algo、--optimize-algo
 - (2) --kl-threshold-size、--divergence-nbins
 - (3) --percentile、--moving-alpha
 - (4) --neg-scale
3. Added Weights quantization algorithm: mle
4. Added weights quantization algorithm parameters
 - (1) --weights-quantize-algo、--weights-optimize-algo
 - (2) --weights-threshold-size、--weights-neg-scale

Issue 1.2 (2022-11-21)

1. This is the Third official release.
1. Update the Ubuntu environment corresponding to the tool version, and add the documentation in the toolkit.
2. Add parameter --dtypes instruction.
3. Added quantized tflite conversion guide.
4. Added confirmation steps for model conversion errors and board measurement accuracy errors in the FAQ chapter.

Issue1.1 (2022-08-30)

This is the second official release.

1. Updated the corresponding version of each Framework
2. Added model conversion flowchart
3. Add conversion instructions for int8 perlayer
4. Add conversion command for Paddle model.

Issue 1.0 (2022-05-30)

This is the initial release.

Contents

Revision History	ii
1. Introduction	1
2. Tools	2
2.1 Overview	2
2.2 Model Converting Tool.....	2
2.2.1 Tool structure	2
2.2.2 Introduction to the Tool Directory.....	2
2.2.3 Environmental Dependencies	5
3. Model Conversion.....	7
3.1 Overview	7
3.2 Quantization.....	7
3.2.1 Introduction.....	7
3.2.2 Quantification method introduction	8
3.2.3 Rules of Selecting a Proper Quantitation Type.....	9
3.3 Model Conversion Flowchart	9
3.4 Model Conversion Procedure	9
3.4.1 Default quantization algorithm	9
3.4.2 Other quantization algorithms	14
3.5 Parameters.....	21
3.5.1 Overview.....	21
3.5.2 adla_convert Parameters	21
4. Other Tool.....	25
4.1 Adla_info	25
4.2 Quantize optimization_tool	26
4.3 NNAPI_Optimize_tool.....	27
4.4 NNIDE_Tool	27

1. Introduction

This document detailly describes the process of how to use Aala_tool to covert models for both internal and external developers.

Currently, Aala_tool supports models corresponding to such as, caffe, pytorch(quantized pytorch/float pytorch), onnx, mxnet, darknet, tensorflow, keras, tflite Paddle and quantized_tflite frameworks. If developer uses another type of model, convert the model type to one of the above types first.

Confidential!
nick@khadas.com
2024-11-20 08:49:13

2. Tools

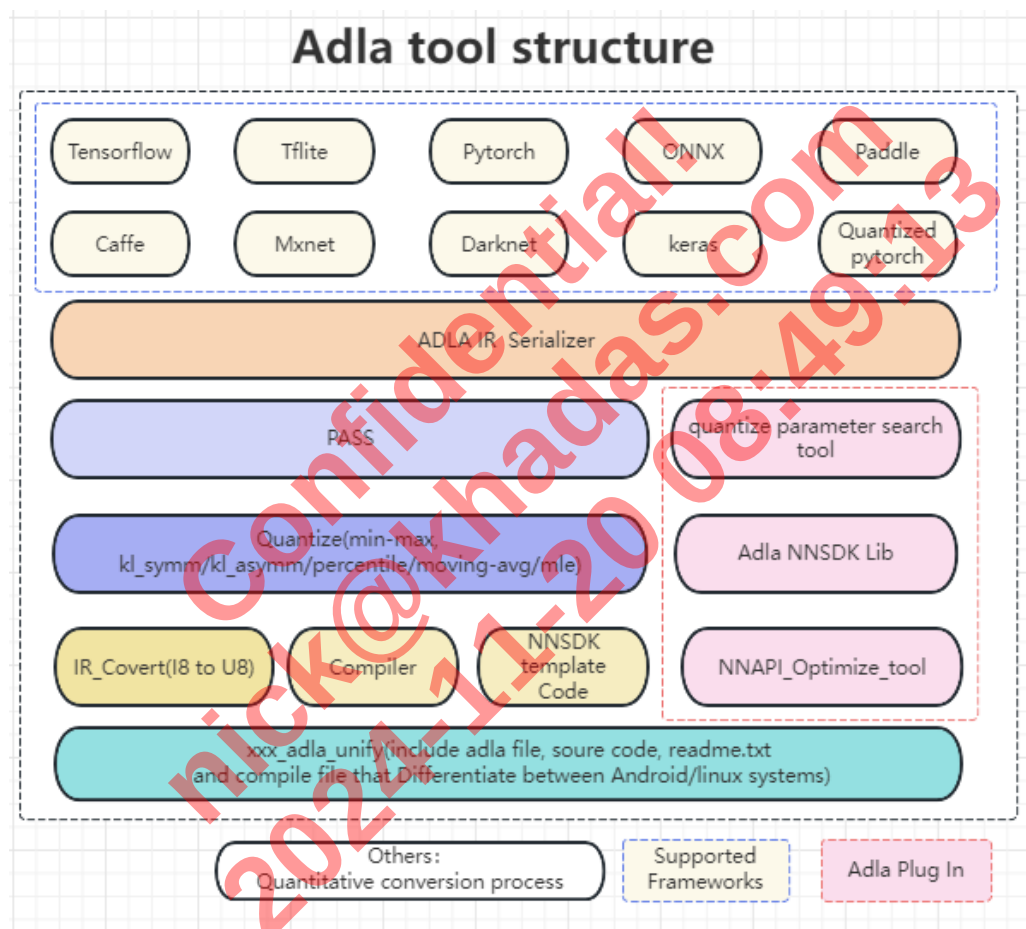
2.1 Overview

Currently, the following tool can be used to convert models to adla files.

- adla-toolkit model: it can be used to import, quantize and convert models to final adla file.

2.2 Model Converting Tool

2.2.1 Tool structure



2.2.2 Introduction to the Tool Directory

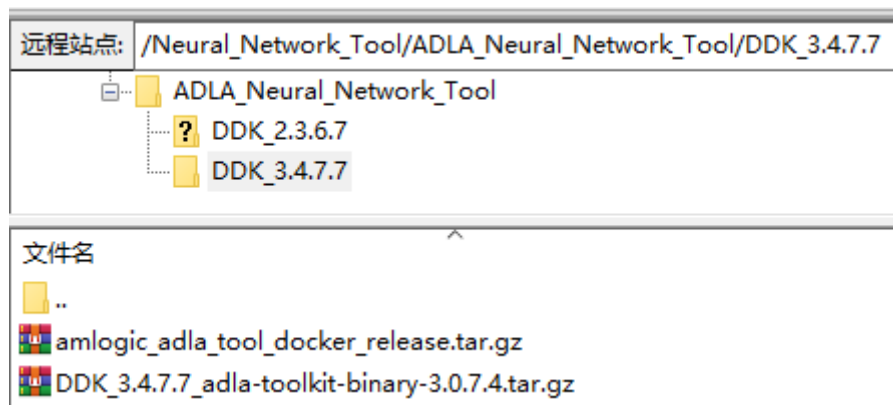
How to obtain tools:

FTP server: ftp-china.amlogic.com

account: amlogic_nn_tool_release.guest

FTP server Path: /Neural_Network_Tool/ADLA_Neural_Network_Tool/

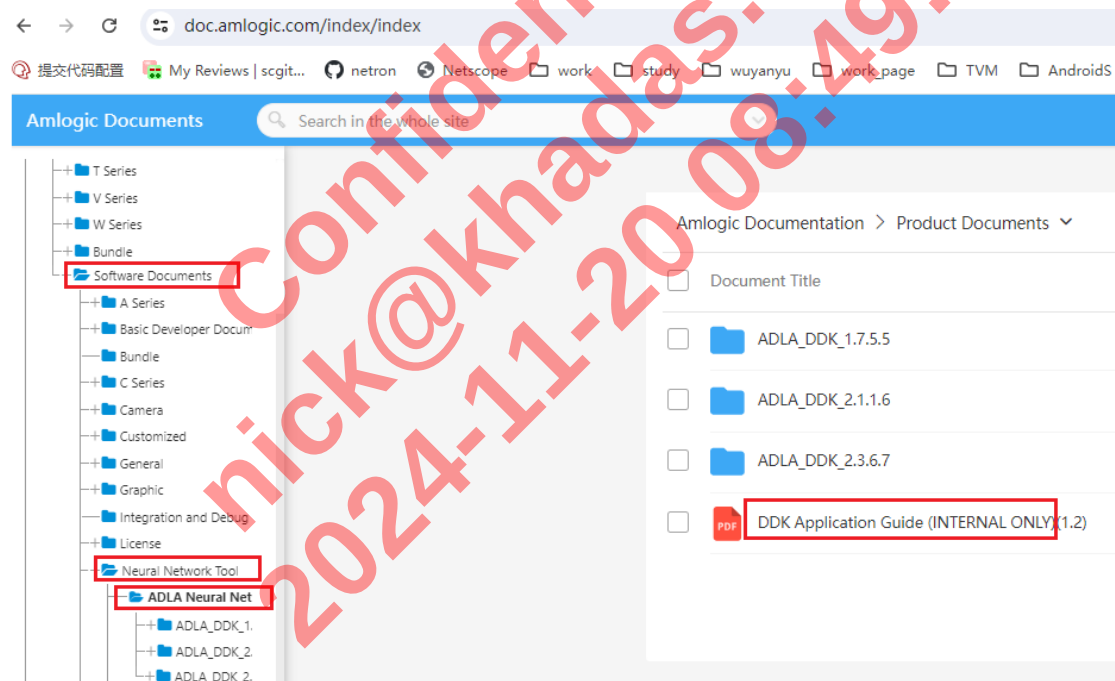
Obtain the tools in the corresponding DDK version directory, such as DDK_3.4.7.7:



Note: Use the filezilla application on the computer and log in with the account and password obtained to obtain the corresponding tools. The port defaults or is set to 21.

adla_tool is released through ftp server. Customers please contact the amlogic docking person to obtain the latest access password from the DDK Application Guide (INTERNAL ONLY).doc in the application document section and provide it to the customer. How to get it is as shown below:

Link: <https://doc.amlogic.com/index/index>



File In Adla_tool_binary-x.x.x	Description
bin/	Directory containing the executable file for adla tool
adla_convert	soft link to -> ./adlplib/adla_convert

adlplib\	Directory containing the executable file for adla_tool
adla_convert	binary conversion command file,convert model to adla format
adla_plug_in\	Some plugins for adla tools
common\	Compiler and Configuration Files
template\	Template code and nnsdk package
android\	Template code for Android environment
linux\	Template code for Linux environment
example\	The demo code of nnsdk, will be readjusted in the future
nnsdk\	nnsdk development kit
tools\	some auxiliary tools
adla_info_tool\	adlainfo tool,parse adla file information
adla_visualization_tool	adla netron tool, adla file visualization tool
Quantize_optimization_tool	Optimize quantization accuracy tools
tflite_refactorer_tool	tflite refactorer tool, Applicable to use NNAPI to integrate apk scenarios
input_npy_txt_bin_generate.py	Generate random input npy/txt/bin file script
demo/	Directory containing the samples for adla tool
convert_adla.sh	samples model conversion script
data/	Model quantization picture
dataset.txt	Model Quantization Dataset
model_source/	Models corresponding to each framework supported by adla
Readme.txt	Readme file
release_notes.txt	Release notes file

2.2.3 Environmental Dependencies

Current version of adal_tool is generated based on Ubuntu20.04 python3.8.10, those are the only environmental requirements. There are no other dependencies.

Correspondence between the driver version on the board side and the tool version:

Restart the board to get the serial port kernel log, or directly execute the command:

dmesg |grep version,

```
5.405204] adla kmd version: 0.10.7.1
```

1. The driver version corresponding to adla is 0.10.7.1, currently only pay attention to the first three.
2. The corresponding relationship between driver, DDK version and adla_tool version is as follows

DDK_Version	Driver Version	Adla_tool Version
DDK 1.2.2.4	0.10.2	adla-toolkit_0.9.3.6/1.0.0.6
DDK 1.4.3.5	0.10.4	adla-toolkit_1.2.0.7
DDK 1.7.5.5	0.10.7	adla-toolkit_1.2.0.9
DDK 2.1.1.6	1.2.0	adla-toolkit_1.5.10 adla-toolkit_1.6.10.3
DDK2.3.6.7	1.3.0	adla-toolkit_2.0.11.0
DDK 3.4.7.7	1.4.0	adla-toolkit_3.0.7.4

Note:

1. Driver version number: the version number obtained in the second step.
2. Adla_tool version: Tool version number, which needs to match the Driver version.
3. DDK_Version: just a DDK version number for external release.

The supported versions of each framework, and model-load methods, are shown as follows:

Framework	Version	Model load(Remark)
Tensorflow	2.15.0	Tf.compat.v1.lite.TFLiteConverter.from_frozen_graph()
Pytorch	2.0.0	torch.jit.load(model_path)
Onnx	1.12.0	onnx.load(model_path)
Mxnet	1.6.0	mx.model.load_checkpoint(prefix_path, prefix_epoch)

Darknet	https://github.com/siju-samuel/darknet	
Caffe	apt-get install caffe-cpu	caffe.Net(proto_file,blob_file,caffe.TEST)
tflite	2.15.0	tflite.Model.GetRootAsModel tflite.Model.Model.GetRootAsModel
Keras	3.0.5	tf.keras.models.load_model(model_path)
Paddle	2.4.2	Paddle.jit.load(model_path)

Confidential!
nick@khadas.com
2024-11-20 08:49:13

3. Model Conversion

3.1 Overview

The model conversion includes three steps. First, convert each framework models into adla IR in a unified format, quantize into int8, int16 and uint8 format ,and then convert it into an adlf files that can be loaded and run on Amlogic platform.

3.2 Quantization

Quantization is the process that stores floating point numbers which used to be stored in 32-bit or 64-bit, in 16/8 or even 4-bit data format instead.

Currently Adla tool multiplex tensorflow for quantization, and supports int8(Perlayer/Perchannel)、int16 and uint8 quantization types.

3.2.1 Introduction

Uint8: 32-bit floating point numbers are quantized into unsigned int8 data. That is, map the values in the maximum/minimum range into 0~255.

Int8: 32-bit floating point numbers are quantized into signed int8 data. That is, map the values in the maximum/minimum range into -128~127.

Int16: Floating point numbers are quantized into signed int16 data. That is, map the values in the maximum/minimum range into -32768~32767.

The calculation formula of int8/uint8 quantization parameters:

$$\text{scale} = (\text{float_max_value} - \text{float_min_value}) / (\text{quantized_max} - \text{quantized_min})$$

$$\text{zero_point} = \text{quantized_max} - \text{float_max_value} / \text{scale}$$

scale: Mapping ratio

zero_point: position of zero point

The calculation formula of int16 quantization parameters:

$$\text{float_max_temp} = \max(|\text{float_max_value}|, |\text{float_min_value}|)$$

$$\text{scale} = \text{float_max_temp} / \text{quantized_max}(32767)$$

$$\text{zero_point} = 0$$

scale: Mapping ratio

For example: the maximum/minimum value of some node point is (-4~1.076).

The uint8 quantization parameters are calculated as follows:

$$\text{scale} = (1.076 - (-4)) / (255 - 0) = 0.0199$$

$$\text{zero_point} = 255 - 1.08 / \text{scale} = 201$$

The int8 quantization parameters are calculated as follows:

$$\text{scale} = (1.076 - (-4)) / (127 - (-128)) = 0.0199215$$

$$\text{zero_point} = 127 - 1.076 / \text{scale} = 73$$

The int16 quantization parameters are calculated as follows:

$$\text{float_max_temp} = \max(|-4|, |1.08|)$$

scale = $4/32767 = 0.000122074$

3.2.2 Quantification method introduction

Model quantization includes Feature quantization and Weights quantization.

Feature quantification method:

1. MinMax quantization (Default): Select the maximum and minimum values directly from the fp32 tensor to confirm the dynamic range
2. kl_symm_divergence: KL symmetric quantization algorithm abstracts the float32 numerical distribution and int8 numerical distribution into two distributions, uses the threshold |T| to update the two numerical distributions, and uses the KL divergence to measure the similarity of the two distributions.
3. Kl_asymm_divergence: KL asymmetric quantization, abstracts float32 numerical distribution and int8 numerical distribution into two distributions, uses [Min,Max] to update these two numerical distributions, and uses a scaling factor scale to narrow the range of [Min,Max], And use the KL divergence to measure the similarity of these two distributions.
4. moving-avg: Moving average maximum and minimum values.
5. Percentile: Adjust the dynamic range of the maximum and minimum values in the fp32 tensor based on a percentage factor. This algorithm is more effective for models with individual extremely large outlier values in the Feature data distribution.
6. hist_percentile: Histogram percentage coefficient quantification algorithm
7. hist_asymm_percentile: Histogram asymmetric percentage coefficient quantization algorithm
8. history_moving-avg: Historical coefficient moving average quantization algorithm

Weights quantization type: per-channel and per-layer.

Weights Per-layer quantization: The filter tensor of operators such as conv2d/Dwconv2d counts a min/max during quantization, and then performs quantization. Per-layer quantization can reduce the DDR memory and bandwidth occupied when running on a single board.

Weights Per-channel quantization: When quantizing the filter tensor of operators such as conv2d/Dwconv2d, the corresponding min/max values are counted and quantized according to the number of filters. Each filter corresponds to a set of quantization parameters. Per-channel quantization can improve the quantized precision. Especially for models with large differences in the filter range distribution of some conv operators, using per-channel quantization can significantly improve the accuracy.

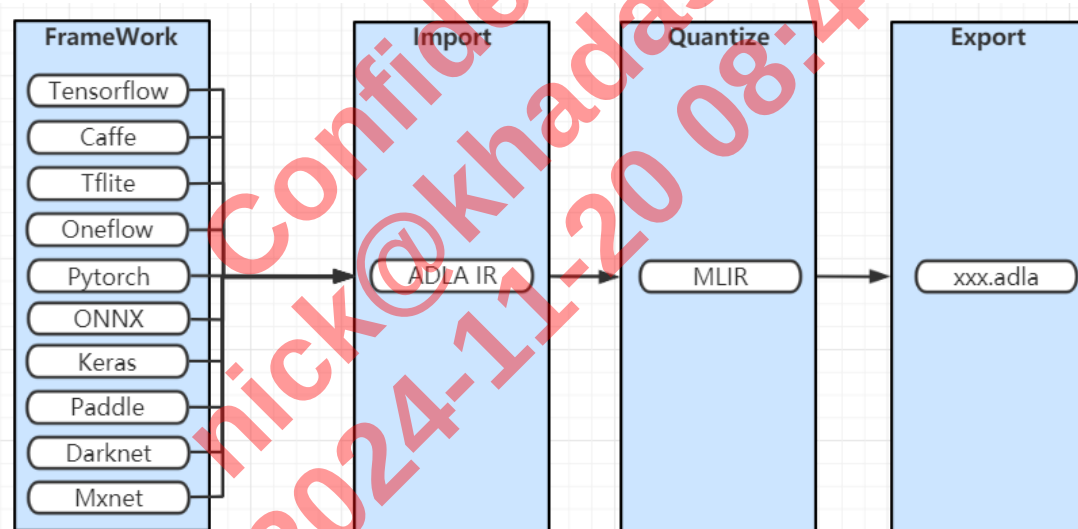
Weights quantization method: There are currently MinMax and MLE quantization methods.

1. MinMax quantization (Default): directly select the maximum and minimum values from the fp32 tensor to confirm the dynamic range
2. MLE: Maximum Likelihood Estimation Quantization Method, valid for both perlayer and perchannel quantization

3.2.3 Rules of Selecting a Proper Quantitation Type

1. Generally, it is recommended to use int8 perlayer quantization. Currently, this is also the default type.
2. If the precision of int8 perlayer quantization is not enough, it can be set to perchannel quantization or use int16 quantization.
3. The current feature and weights both use the MinMax quantization algorithm by default. The Feature algorithm and the weights algorithm can improve the accuracy of the model under certain parameter conditions, and the weights and feature quantization algorithms can increase the accuracy by about 0.5% on average. The parameters corresponding to the optimal quantization accuracy of each model are inconsistent. Customers need to traverse the quantization parameters and verify the accuracy after quantization to find the optimal result. The combination of the Feature+weights algorithm also needs to traverse more parameters to improve the accuracy.
4. The selection of the feature quantization algorithm has nothing to do with the per-layer/perchannel quantization of weights

3.3 Model Conversion Flowchart



3.4 Model Conversion Procedure

Model conversion is done in the adla-toolkit directory. The executable files are all under bin directory.

3.4.1 Default quantization algorithm

- 1、Feature/weights Min-Max quantization:

Caffe/Darknet:

Int8/Int16:

```
/bin/adla_convert --model-type caffe(darknet) \
```

```

--model          xx.prototxt(xxx.cfg) \
--weights        xx.caffemodel(xxx.weights) \
--quantize-dtype  int8(int16/uint8) \
--source-file     dataset.txt \
--channel-mean-value "128,128,128,128" \
--outdir         convert_output \
--target-platform PRODUCT_PID0XA001
--disable-per-channel False(default True) \
--export-template-code True(default True) \
--system-env 'android' \
#--batch-size    4(default 1)

```

Onnx/Tflite:**Int8/Int16:**

```

/bin/adla_convert --model-type onnx(tflite) \
--model          ./xxxx.onnx(xxx.tflite) \
--quantize-dtype  int8(int16/uint8) \
--inputs         "0 1 2" \
--input-shapes    "3,288,288#3,288,288#3,288,288" \
--source-file     dataset.txt \
--channel-mean-value "128,128,128,128#0,0,0,255#0,0,0,255" \
--outdir         convert_output \
--target-platform PRODUCT_PID0XA001 \
--batch-size      1
--disable-per-channel False(default True) \
--export-template-code True(default True) \
--system-env 'android' \
--outputs "output0 output1" \ #onnx/tflite supports specifying output nodes
#-shape-with-batch False#False#False
#-dtypes      "float32#float32#float32"

```

Mxnet:**Int8/Int16:**

```

/bin/adla_convert --model-type mxnet \
--model          ./mobilenet0.25-symbol.json \
--weights        ./mobilenet0.25-0000.params \
--quantize-dtype  int8(int16/uint8) \
--inputs         "data" \
--input-shape     "3,224,224" \
--channel-mean-value "0,0,0,255" \
--source-file     dataset.txt \
--outdir         convert_output \
--target-platform PRODUCT_PID0XA001
--export-template-code True(default True) \
--system-env 'android' \
#--batch-size    4(default 1)

```

Tensorflow:**Int8/Int16:**

```

/bin/adla_convert --model-type tensorflow \
--model          xxxx.pb \
--inputs         input1 \
--quantize-dtype  int8(int16/uint8) \
--input-shape     112,112,3 \
--outputs         fc1/mul \
--channel-mean-value "0,0,0,255" \
--source-file     dataset.txt \
--outdir         convert_output \
--target-platform PRODUCT_PID0XA001 \
--export-template-code True(default True) \

```

```
--system-env 'android' \
```

Keras:

Int8/Int16:

```
/bin/adla_convert --model-type keras \
--model xxx.h5 \
--quantize-dtype int8(int16) \
--input-shape "224,224,3" \
--channel-mean-value "0,0,0,255" \
--source-file dataset.txt \
--outdir convert_output \
--target-platform PRODUCT_PID0XA001
--export-template-code True(default True) \
--system-env 'android' \
```

Quantized Tflite:

```
./bin/adla_convert --model-type quantized_tflite\
--model resnet18.tflite \
--target-platform PRODUCT_PID0XA001 --outdir test
```

Pytorch:

Int8/Int16:

```
./bin/adla_convert --model-type pytorch \
--model xxx.pt \
--quantize-dtype int8(int16/uint8) \
--inputs data \
--input-shape 3, 224,224 \
--channel-mean-value "0,0,0,255" \
--source-file dataset.txt \
--outdir convert_output \
--target-platform PRODUCT_PID0XA001
--export-template-code True(default True) \
--system-env 'android' \
```

Paddle:

Int8/Int16:

```
./bin/adla_convert --model-type paddle \
--model xxxxx \
--quantize-dtype int8(int16/uint8) \
--inputs data \
--input-shape 224,224,3 \
--channel-mean-value "0,0,0,255" \
--source-file dataset.txt \
--outdir convert_output \
--target-platform PRODUCT_PID0XA001
--export-template-code True(default True) \
--system-env 'android' \
```



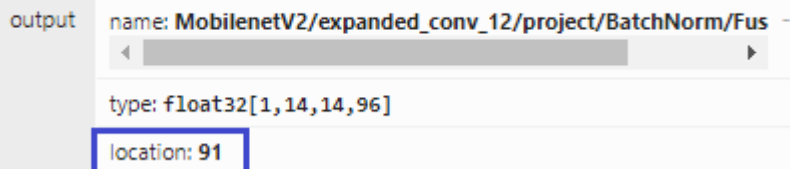
Note

1. **--model-type**: Model type used in the conversion. Currently it supports onnx, pytorch, caffe, darknet, mxnet, keras, quantized_tflite, tflite, tensorflow, paddle etc.
2. **--model(model files)/--weights(model weights file)**: **--model** parameter must be set. **--weights** needs to be set when **model-type** is caffe/darknet/mxnet.
3. **--inputs/--input-shapes**: It needs to be set when the model-dtype is not quantized_tflite/caffe/darknet.
4. **--inputs**: Input node names. If there are multiple inputs, use the space as the separator. For example, "input1 input2".

5. **--output:** Output node names. If there are multiple outputs, use the space as the separator. For example, "output1 output2". When **model-type** is tensorflow, **outputs** needs to be set. When the model-type is onnx/tflite, it does not need to be set by default, but it supports setting the middle layer as an output node.

The Onnx model can directly set the name corresponding to the OUTPUTS of the middle layer. The Tflite model sets the location number corresponding to the middle layer OUTPUTS. Use the Netron tool to view the location number corresponding to the OUTPUTS name of the onnx model and the OUTPUTS of the tflite model, and the tflite locat number is as follows:

OUTPUTS



6. **--input-shapes:** The sizes of input nodes. If there are multiple inputs, use "#" as the separator. For example, "3,224,224#3,299,299#3,12,12".
7. **--shape-with-batch:** Indicates whether the input shape carries the batch information or not. This parameter is optional. The default value is False. If the value is True, the size of the actual input of the model is set by **input-shapes**. If there are multiple inputs, use "#" as the separator. For example, when the model inputs are "1x224x224x3", "11x88", then **input-shape** is set to "224,224,3#11,88", **shape-with-batch** is set to "False#True".
8. **--dtypes:** Input type, set the type information corresponding to the input (optional). Default: float32. If there are multiple inputs, please separate them with #, for example: --dtypes : "float32#float32#int8"
9. **--quantize-dtype:** quantization type. Currently, it supports int8, int16 and uint8 quantization types.

When quantization type is set to int8/int16, --source-file/--channel-mean-value need to be set. But if source-file is npy file, channel-mean-value does not need to be set. When quantization type is uint8, --mean/--std-value/--default-ranges-min/--default-ranges-max need to be set.

10. **--source-file:** dataset.txt. The txt file contains paths to the images that need to be quantitated. it supports image and npy files. For example, ./data/cat.jpg or ./data/input_3_224_224.npy.

Generally, it is recommended to provide 500~1000 images for quantitating. If there are multiple inputs in one group, Images which belong to the same group are put in the same line of dataset.txt, and use space as the separator. For example, ./1.jpg ./2.jpg ./3.jpg. if there are multiple groups. Each group is put in one line. When npy files are used as the quantitative data, **shape** needs to be consistent with **input-shapes**.

11. **--channel-mean-value** the pre-processing parameters are set according to the pre-processing methods used during model training. It includes four values, m1, m2, m3, and scale. The first three are mean-value parameters and the last one is the scale parameter. For the input data with three channels (data1, data2, data3), the pre-processing steps are:

$$\text{Out1} = (\text{data1} - m1) * \text{scale}$$

$$\text{Out2} = (\text{data2} - m2) * \text{scale}$$

$$\text{Out3} = (\text{data3} - m3) * \text{scale}$$

For example, if the pre-processing requires normalizing the data into the range of [-1, 1], set the parameter values as 128, 1228, 128, 128.

If the pre-processing requires normalizing the data into the range of [0, 1], set the parameter values as 0, 0, 0, 255.

For single-channel model, set the parameter values as m1, 0, 0, scale.

For Multiple-input model, different **channel-mean-value** can be set for inputs, using # as the separator. For example, `--channel-mean-value "0,0,0,1#127.5,127.5,127.5,127.5#0,0,0,255"`

If the scales of the three channels of an image are different, or A model is not an image, it is recommended to read the model input data and save it as a npy file after subtracting mean and dividing by scale.

It is recommended to use Python to read the input data of the model, subtract it by mean-value and divide it by scale, and save it as a npy file. During model conversion, Provide the npy file path in the quantitated source-file.

12. **--batch-size**: Set the batch-size for the adla file after conversion. Currently, the default value is 1.
13. **--iterations**: Optional parameter,default value 1000. if dataset.txt provides data for multi-groups, and needs to use all data provided for quantization, set `--iterations` and make sure `iterations*batch-size=` Number of data in the multi-groups.
14. **--disable-per-channel**: Whether to disable per-channel quantization, the default is True.When set to True, it is perlayer quantized, otherwise it is perchannel quantized
15. **--outdir**: Directory for the generated, the default value is the "xxx_adla_unify" directory under the current path.
16. **--target-platform**: specify the target platform for the adla file. Currently, the default value is `PRODUCT_PID0XA001`.

There are currently valid values:

`PRODUCT_PID0XA001`、`PRODUCT_PID0XA002`

`PRODUCT_PID0XA003`、`PRODUCT_PID0XA004`

`PRODUCT_PID0XA005`

You can use the following command to confirm the value that should be set.Execute:
`cat /proc/cpuinfo` or `cat /proc/cpu_chipid`

View the first four characters of the sequence code corresponding to Serial in the second last line (the part in bold after Serial)

Serial number and parameters `PRODUCT_PID0X?` with the relationship are as follows:

PID	Serial
0XA001	Serial: 3d0a 010100000000c613492931563343
0XA002	Serial: 3e0a 0202000000007c404b6c31563553
0XA003	Serial: 360c 01030000000025f904ab31563541
0XA004	Serial: 420a 010100000000daff6e4931583354
0XA005	Serial: 480a 0201000000006827632931563653

For example:

After executing the command in the serial port or cmd window: `cat /proc/cpuinfo`.Serial is as shown below:

```
Serial      : 360c010300000000946bd13930563754
Hardware    : Amlogic
```

At this point, the parameter should be set: `--target-platform PRODUCT_PID0XA003`

Note:

1. If you have not obtained the serial number corresponding to the above table, you can execute: `cat /proc/cpu_chipid`
2. If the first four digits of the serial number do not match, you can only match the first three digits
17. `--export-template-code`: Whether to export demo code, the default is `True`. When set to `False`, only adla files will be converted.
18. `--system-env`: Specify the single board system environment, the default is `linux`. Use with `--export-template-code`. When the default parameters are used, the exported demo code contains the compiled script corresponding to `linux`. When it is set to `"android"`, the exported demo code contains the compiled script corresponding to `Android`. Currently supports `[linux, android]` two effective values, the user can set according to the current board environment. Then refer to the guidance of `Readme.txt` in the generated demo code to compile.

Result: Finally, the demo code with adla file is generated, as shown in the figure below:

```
resnet34-v2-7_adla_unify/
├── CMakeLists.txt
├── config.h
├── main.c
├── Readme.txt
└── resnet34-v2-7_int8.adla
```

3.4.2 Other quantization algorithms

Generally, the default quantization algorithm in the previous chapter can be used. If the accuracy of the default quantization algorithm does not meet your needs, you can try using the advanced quantization algorithm.

Currently, it is adapted to a variety of quantization algorithms, and the parameters corresponding to different quantization algorithms are different. The following is a list of how to use different quantization algorithms. If customers need to use advanced quantization algorithms, it is recommended that customers refer to the `Quantize_optimization_tool` introduced in Chapter 4.2 to use advanced quantization algorithms.

2、Feature `kl_symm_divergence+weights` Min-max quantization

Take the `caffe` model as an example

```
./bin/adla_convert --model-type caffe(darknet) \
  --model xx.prototxt(xxx.cfg) \
  --weights xx.caffemodel(xxx.weights) \
  --quantize-dtype int8(int16/uint8) \
  --source-file dataset.txt \
  --channel-mean-value "128,128,128,128" \
  --outdir convert_output \
  --target-platform PRODUCT_PID0XA001 \
  --disable-per-channel False \
  --quantize-algo "kl_symm_divergence" \
  --optimize-algo "kl-dis" \
  --kl-threshold-size 8192 \
  --divergence-nbins 16384 \
```

```
--export-template-code True(default True) \
--system-env 'android'
```

Note:

1. Compared with the default MinMax quantization, KL symmetric quantization adds four new parameters, and the settings of other parameters are consistent with MinMax quantization
2. --quantize-algo: The default value is: "min-max", the current setting quantization algorithm type is "kl_symm_divergence".
3. --optimize-algo: Method to measure the similarity of float32 numerical distribution and int8 numerical distribution. Default value: "kl-dis", when kl_symm_divergence is set, the corresponding valid value is: "kl-dis"/"mse-dis".
4. --kl-threshold-size: Whether the Feature uses kl quantization threshold when KL quantization. If the quantized Feature size is too small, KL quantization should not be used. Default value: 2048, it is generally recommended to use the default value directly.
5. --divergence-nbins:KL 量化时，直方图中 bin 的个数。默认值：2048，建议取值:2048/4096/8192/12288/16384.

Some models use the kl symmetric quantization algorithm, and the accuracy after quantization with different parameters is shown in the figure below: (the first line of results is the corresponding accuracy of min-max quantization).

quantize_algo	optimize_algo	num_histogram_bins	kl_threshold_size	resnet_18		resnet_34		googlenet-7		shufflenet-v2-10	
				TOP1	TOP5	TOP1	TOP5	TOP1	TOP5	TOP1	TOP5
NA	NA	NA	NA	62.72%	84.64%	67.24%	88.22%	67.50%	88.20%	49.98%	72.98%
kl_symm_divergence	kl-dis	2048	2048	63.22%	84.66%	67.46%	88.24%	67.36%	88.44%	46.06%	70.60%
			4096	63.26%	84.54%	67.82%	88.28%	67.64%	88.20%	46.14%	70.72%
			8192	62.60%	84.72%	67.12%	88.00%	67.56%	88.40%	46.36%	70.82%
			12288	62.60%	84.72%	67.12%	88.00%	67.28%	88.36%	46.62%	72.08%
			16384	62.60%	84.72%	67.12%	88.00%	67.34%	88.36%	46.62%	72.08%
		2048	2048	62.92%	84.74%	67.18%	87.96%	67.22%	88.38%	45.38%	69.50%
			4096	62.38%	84.70%	66.78%	87.88%	67.20%	88.36%	50.30%	72.86%
			8192	62.38%	84.70%	66.78%	87.88%	67.12%	88.34%	50.30%	72.86%
			12288	62.38%	84.70%	66.78%	87.88%	66.96%	88.42%	49.84%	72.88%
			16384	62.38%	84.70%	66.78%	87.88%	67.20%	88.32%	49.84%	72.88%
		4096	2048	62.52%	84.72%	67.14%	88.00%	67.16%	88.28%	50.30%	72.86%
			4096	62.52%	84.72%	67.14%	88.00%	67.40%	88.32%	50.30%	72.86%
			8192	62.52%	84.72%	67.14%	88.00%	67.22%	88.24%	50.30%	72.86%
			12288	62.52%	84.72%	67.14%	88.00%	67.20%	88.16%	49.84%	72.88%
			16384	62.52%	84.72%	67.14%	88.00%	67.26%	88.28%	49.84%	72.88%
	mse-dis	8192	2048	62.60%	84.76%	67.10%	88.12%	67.34%	88.32%	49.66%	72.64%
			4096	62.60%	84.76%	67.10%	88.12%	67.16%	88.12%	49.66%	72.64%
			8192	62.60%	84.76%	67.10%	88.12%	67.42%	88.20%	49.66%	72.64%
			12288	62.60%	84.76%	67.10%	88.12%	67.28%	88.06%	49.86%	73.04%
			16384	62.60%	84.76%	67.10%	88.12%	67.34%	88.18%	49.86%	73.04%
		12288	2048	62.86%	84.74%	67.12%	87.92%	67.40%	88.20%	50.16%	72.66%
			4096	62.86%	84.74%	67.12%	87.92%	67.48%	88.24%	50.16%	72.66%
			8192	62.86%	84.74%	67.12%	87.92%	67.26%	88.30%	50.16%	72.66%
			12288	62.86%	84.74%	67.12%	87.92%	67.38%	88.22%	49.94%	73.00%
			16384	62.86%	84.74%	67.12%	87.92%	67.40%	88.34%	49.94%	73.00%
		16384	2048	62.68%	84.56%	67.00%	88.08%	67.06%	88.16%	49.94%	72.86%
			4096	62.68%	84.56%	67.00%	88.08%	67.42%	88.36%	49.94%	72.86%
			8192	62.68%	84.56%	67.00%	88.08%	67.14%	88.06%	49.94%	72.86%
			12288	62.68%	84.56%	67.00%	88.08%	67.10%	88.22%	50.14%	72.96%
			16384	62.68%	84.56%	67.00%	88.08%	67.46%	88.30%	50.14%	72.96%

3、Feature Kl_asymm_divergence+weights Min-max quantization

Take the caffe model as an example

```
./bin/adla_convert --model-type caffe(darknet) \
--model xx.prototxt(xxx.cfg) \
```

```

--weights                xx.caffemodel(xxx.weights) \
--quantize-dtype          int8(int16/uint8) \
--source-file            dataset.txt \
--channel-mean-value     "128,128,128,128" \
--outdir                 convert_output \
--target-platform        PRODUCT_PID0XA001 \
--disable-per-channel    False \
--quantize-algo           "kl_asymm_divergence" \
--optimize-algo          "kl-dis" \
--kl-threshold-size      2048 \
--divergence-nbins       2048 \
--neg-scale              0.9 \
--export-template-code   True(default True) \
--system-env 'android'

```

Note:

1. Compared with the default MinMax quantization, KL symmetric quantization adds four new parameters, and the settings of other parameters are consistent with MinMax quantization
2. --quantize-algo: The default value is: "min-max", the current setting quantization algorithm type is "kl_symm_divergence"
3. --optimize-algo: Method to measure the similarity of float32 numerical distribution and int8 numerical distribution. Default value: "kl-dis", when kl_symm_divergence is set, the corresponding valid value is: "kl-dis"/"js-dis"/"cos-dis"
4. --kl-threshold-size: Whether the Feature uses kl quantization threshold when KL quantization. If the quantized Feature size is too small, KL quantization should not be used. Default value: 2048, it is generally recommended to use the default value directly.
5. --divergence-nbins: KL quantization, the number of bins in the histogram. Default value: 2048, recommended value: 2048/4096/8192/12288/16384
6. --neg-scale: Default: 1.0

The precision results of some models quantized using the percentile algorithm of Feature are as follows: (the first row of data is the feature min-max precision)

quantize_algo	optimize_algo	histogram_bin	kl_threshold_size	neg_scale	Number of pictures for	resnet_18		shufflenet_v2-10		squeezenet1.1-7		mnasnet1.0		Darknet19	
NA	NA	NA	NA	NA	200	TOP1	TOP5	TOP1	TOP5	TOP1	TOP5	TOP1	TOP5	TOP1	TOP5
kl_asymm_divergence	cos-dis	2048	2048	0	200	62.72%	84.64%	48.98%	72.98%	50.14%	74.92%	48.68%	73.16%	72.92%	90.92%
				0.001	200	62.98%	84.74%	49.56%	73.02%	50.40%	74.98%	48.58%	73.20%	73.24%	91.38%
				0.01	200	63.08%	84.96%	49.68%	73.04%	50.06%	74.50%	48.74%	73.32%	73.28%	91.20%
				0.1	200	62.92%	84.66%	49.44%	72.96%	50.06%	74.50%	48.66%	73.20%	73.26%	91.30%
				1	200	62.96%	84.70%	49.92%	73.20%	50.06%	74.50%	48.58%	73.24%	73.26%	91.06%
				1.2	200	63.08%	84.96%	50.00%	72.82%	50.06%	74.50%	48.94%	73.12%	73.66%	91.40%
				1.5	200	62.96%	84.58%	49.82%	73.10%	50.06%	74.50%	48.66%	73.12%	73.42%	91.48%
				1.5	200	63.08%	84.96%	49.58%	72.86%	50.06%	74.50%	48.88%	73.30%	73.50%	91.46%
			4096	0	200	62.98%	84.74%	49.78%	73.14%	50.54%	74.84%	48.84%	73.16%	73.30%	91.30%
				0.001	200	62.84%	84.76%	49.84%	72.98%	50.06%	74.50%	48.66%	73.04%	73.50%	91.24%
				0.01	200	63.08%	84.96%	49.92%	72.94%	50.06%	74.50%	48.78%	73.34%	73.22%	91.36%
				0.1	200	63.08%	84.96%	49.82%	72.98%	50.06%	74.50%	49.14%	73.16%	73.40%	91.14%
				1	200	63.10%	84.66%	49.62%	73.20%	50.06%	74.50%	48.88%	73.16%	73.34%	91.24%
				1.2	200	63.08%	84.96%	49.82%	72.94%	50.06%	74.50%	49.10%	73.34%	73.42%	91.34%
				1.5	200	63.08%	84.96%	50.00%	73.14%	50.06%	74.50%	48.88%	73.34%	73.34%	91.36%
			8192	0	200	62.96%	84.64%	50.14%	72.96%	50.72%	75.08%	48.64%	73.32%	73.36%	91.20%
				0.001	200	63.02%	84.78%	50.18%	72.88%	50.06%	74.50%	48.80%	73.32%	73.28%	91.36%
				0.01	200	63.08%	84.96%	49.90%	73.12%	50.06%	74.50%	48.76%	73.24%	73.28%	91.30%
				0.1	200	63.08%	84.96%	49.88%	72.88%	50.06%	74.50%	48.52%	73.22%	73.42%	91.06%
				1	200	63.08%	84.96%	49.90%	72.92%	50.06%	74.50%	48.94%	73.38%	73.16%	91.30%
				1.2	200	63.34%	84.80%	50.12%	72.98%	50.06%	74.50%	48.90%	73.28%	73.40%	91.26%
				1.5	200	63.08%	84.96%	49.66%	73.02%	50.06%	74.50%	48.68%	73.38%	73.36%	91.28%

4、Feature Percentile+weights Min-max quantization

Take the caffe model as an example

```
./bin/adla_convert --model-type  caffe(darknet) \
--model          xx.prototxt(xxx.cfg) \
--weights        xx.caffemodel(xxx.weights) \
--quantize-dtype  int8(int16\uint8) \
--source-file     dataset.txt \
--channel-mean-value "128,128,128,128" \
--outdir          convert_output \
--target-platform PRODUCT_PID0XA001 \
--disable-per-channel False \
--quantize-algo    "percentile" \
--percentile      0.9999 \
--export-template-code True(default True) \
--system-env      'android'
# --asymm-percentile 0.999
```

Note:

1. Compared with the default MinMax quantization, Percentile quantization adds two new parameters, and the settings of other parameters are consistent with MinMax quantization.
2. --quantize-algo: The default value is: "min-max", the current setting quantization algorithm type is "percentile".
3. --percentile: percentage coefficient, default value: 0.9999, recommended value range: 0.99 ~ 1.0.
4. --asymm-percentile: asymmetric percentage coefficient, default value: -1.0, recommended value range: 0.99 ~ 1.0.

When it is the default value, it is a symmetric percentage quantization algorithm. When set to other values, it is an asymmetric percentage quantization algorithm.

The precision results of some models quantized using the percentile algorithm of Feature are as follows: (the first row of data is the feature min-max precision)

quantize_algo	optimize_algo	num_histogram_bins	kl_thread_size	number of pictures for Quant	resnet_18		resnet_34		googlenet-7		shufflenet-v2-10		squeezenet1.1-7	
					TOP1	TOP5	TOP1	TOP5	TOP1	TOP5	TOP1	TOP5	TOP1	TOP5
NA	NA	NA	NA	100	62.72%	84.64%	67.24%	88.22%	67.50%	88.20%	49.98%	72.98%	50.14%	74.92%
percentile	NA	NA	0.9991	100	62.84%	84.62%	66.92%	87.88%	67.02%	88.38%	48.86%	72.62%	50.40%	74.80%
	NA	NA	0.9992	100	62.98%	84.68%	66.76%	87.90%	67.08%	88.28%	48.72%	72.60%	50.22%	74.64%
	NA	NA	0.9993	100	62.80%	84.74%	66.88%	88.12%	67.18%	88.38%	48.74%	72.64%	50.42%	74.70%
	NA	NA	0.9994	100	62.92%	84.70%	66.98%	87.96%	67.16%	88.30%	49.44%	72.66%	50.60%	74.66%
	NA	NA	0.9995	100	63.16%	84.72%	67.14%	87.94%	67.26%	88.28%	49.52%	73.06%	50.18%	75.06%
	NA	NA	0.9996	100	63.04%	84.62%	67.10%	88.10%	67.22%	88.50%	49.40%	72.60%	50.50%	74.78%
	NA	NA	0.9997	100	63.20%	84.64%	67.18%	88.06%	67.24%	88.40%	49.86%	73.14%	50.38%	74.32%
	NA	NA	0.9998	100	63.06%	84.66%	66.98%	88.06%	67.62%	88.24%	49.58%	72.84%	50.12%	74.50%
	NA	NA	0.9999	100	62.88%	84.72%	66.96%	88.36%	67.48%	88.32%	49.82%	72.98%	50.04%	74.68%

5、Feature Moving-avg+weights Min-max quantization

Take the caffe model as an example

```
./bin/adla_convert --model-type  caffe(darknet) \
--model          xx.prototxt(xxx.cfg) \
--weights        xx.caffemodel(xxx.weights) \
```

```

--quantize-dtype      int8(int16/uint8) \
--source-file         dataset.txt \
--channel-mean-value  "128,128,128,128" \
--outdir              convert_output \
--target-platform     PRODUCT_PID0XA001 \
--disable-per-channel False \
--quantize-algo        "moving-avg" \
--moving-alpha         0.2 \
--export-template-code True(default True) \
--system-env 'android'

```

Note:

1. Compared with the default MinMax quantization, moving-avg quantization adds two new parameters, and the settings of other parameters are consistent with MinMax quantization.
2. --quantize-algo: The default value is: "min-max", the current setting quantization algorithm type is "moving-avg".
3. --moving-alpha: moving-alpha coefficient, default value: 0.2, recommended value range: 0.2~0.9.

Part of the model is quantified using the moving-avg algorithm of the Feature, and the accuracy results are as follows: (the first line of data is the feature min-max accuracy)

quantize_algo	optimize_algo	num_histogram_bins	kl_thread_size	number of pictures for Quant	resnet_18		resnet_34		googlenet-7		shufflenet-v2-10		squeezeNet1.1-7	
NA	NA	NA	NA	100	TOP1	TOP5	TOP1	TOP5	TOP1	TOP5	TOP1	TOP5	TOP1	TOP5
moving-avg	NA	0.2	NA	100	62.72%	84.64%	67.24%	88.22%	67.50%	88.20%	49.98%	72.98%	50.14%	74.92%
	NA	0.3	NA	100	60.46%	84.56%	64.80%	88.18%	67.08%	88.30%	49.08%	73.18%	50.34%	74.34%
	NA	0.4	NA	100	61.30%	84.70%	65.54%	88.28%	67.10%	88.42%	49.60%	72.64%	50.22%	74.44%
	NA	0.5	NA	100	61.42%	84.50%	66.00%	88.12%	67.06%	88.26%	49.66%	73.00%	50.78%	74.28%
	NA	0.6	NA	100	61.78%	84.78%	66.08%	87.98%	67.28%	88.44%	49.78%	72.82%	49.94%	73.82%
	NA	0.7	NA	100	62.16%	84.54%	66.46%	87.96%	67.46%	88.32%	49.76%	73.12%	50.06%	74.76%
	NA	0.8	NA	100	62.42%	84.70%	66.82%	87.98%	67.40%	88.42%	49.98%	72.72%	50.42%	74.44%
	NA	0.9	NA	100	62.80%	84.62%	66.56%	87.98%	67.32%	88.52%	50.28%	72.98%	50.00%	74.26%
					62.86%	84.54%	67.00%	88.00%	67.32%	88.30%	49.72%	73.36%	49.98%	74.82%

6、Feature history_moving-avg + weights Min-max quantification

Take the caffe model as an example

```

./bin/adla_convert --model-type      caffe(darknet) \
--model            xx.prototxt(yyy.cfg) \
--weights          xx.caffemodel(yyy.weights) \
--quantize-dtype    int8(int16/uint8) \
--source-file       dataset.txt \
--channel-mean-value "128,128,128,128" \
--outdir            convert_output \
--target-platform   PRODUCT_PID0XA001 \
--disable-per-channel False \
--quantize-algo      "history_moving-avg" \
--moving-alpha       0.2 \
--export-template-code True(default True) \
--system-env 'android'

```

Note:

1. The parameters of history_moving-avg are consistent with the parameters of the moving-avg algorithm.

2. *--quantize-algo: The default value is: "min-max", and the current quantization algorithm type is set to "history_moving-avg".*

7、Feature hist_percentile/hist_asymm_percentile + weights Min-max quantification

Take the caffe model as an example

```
./bin/adla_convert --model-type caffe(darknet) \
--model xx.prototxt(xxx.cfg) \
--weights xx.caffemodel(xxx.weights) \
--quantize-dtype int8(int16/uint8) \
--source-file dataset.txt \
--channel-mean-value "128,128,128,128" \
--outdir convert_output \
--target-platform PRODUCT_PID0XA001 \
--disable-per-channel False \
--quantize-algo "hist_percentile" \ # "hist_asymm_percentile"
--percentile 0.9999 \
--export-template-code True(default True) \
--system-env 'android'
```

Note:

1. The parameters of hist_percentile/hist_asymm_percentile are the same. You only need to set the percentage coefficient --percentile.
2. --quantize-algo currently sets the quantization algorithm type to "hist_percentile" or "hist_asymm_percentile"

8、Feature Min-Max+Weights MLE quantification

Take the caffe model as an example

```
./bin/adla_convert --model-type caffe(darknet) \
--model xx.prototxt(xxx.cfg) \
--weights xx.caffemodel(xxx.weights) \
--quantize-dtype int8(int16/uint8) \
--source-file dataset.txt \
--channel-mean-value "128,128,128,128" \
--outdir convert_output \
--target-platform PRODUCT_PID0XA001 \
--disable-per-channel True \
--weights-quantize-algo "mle" \
--weights-optimize-algo "cos-dis" \
--weights-threshold-size 2048 \
--weights-neg-scale 0.999 \
--export-template-code True(default True) \
--system-env 'android'
```

Note:

1. Compared with the default MinMax quantization of weights, four new parameters have been added for mle quantization. The settings of other parameters are consistent with MinMax quantization, but `--disable-per-channel True` is required, that is, the default value of the tool can be used for this parameter.
2. `--weights-quantize-algo`: The default value is: "min-max", the current setting quantization algorithm type is "mle".
3. `--weights-optimize-algo`: default value (cos-dis), the current valid value is: "cos-dis"/"kl-dis"/"js-dis".
4. `--weights-neg-scale`: The scaling factor when weights are quantized, the default value (0.999), the current settable range: 0.99~1..

The accuracy results of some models quantized using Weights' MLE algorithm are as follows: (the first row of data is min-max accuracy).

Weights quantize algo	optimize algo	weights thread_size	Number of pictures for Quantize	wegiths neg_scale	resnet18-v1-7		resnet34-v2-7		resnet50-v2-7		shufflenet-v2-10		squeezenet1.1-7	
Min-Max	NA	NA	200	NA	TOP1	TOP5	TOP1	TOP5	TOP1	TOP5	TOP1	TOP5	TOP1	TOP5
MLE	cos-dis	2048	200	0.99	64.70%	87.52%	70.88%	89.10%	71.62%	89.18%	51.54%	74.42%	50.60%	74.70%
				0.991	64.98%	87.26%	71.28%	89.90%	71.60%	90.30%	51.36%	74.14%	49.44%	74.58%
				0.999	65.22%	87.44%	70.84%	89.94%	72.12%	90.12%	49.92%	72.98%	49.32%	74.18%
	js-dis	16384	200	0.99	64.88%	87.48%	71.12%	89.12%	71.54%	89.90%	51.60%	73.98%	49.98%	74.10%
				0.99	64.88%	87.48%	70.94%	89.94%	71.54%	89.90%	49.94%	72.50%	49.98%	74.10%
				0.99	64.88%	87.48%	70.94%	89.94%	71.54%	89.90%	51.96%	74.08%	49.98%	74.10%
	kl-dis	2048	200	0.99	64.88%	87.48%	70.94%	89.94%	71.54%	89.90%	49.94%	72.50%	49.98%	74.10%
				0.99	64.88%	87.48%	70.94%	89.94%	71.54%	89.90%	49.94%	72.50%	49.98%	74.10%

9、Feature kl_asymm_divergence + Weights MLE quantization

Take the caffe model as an example

```
./bin/adla_convert --model-type caffe(darknet) \
--model xx.prototxt(xxx.cfg) \
--weights xx.caffemodel(xxx.weights) \
--quantize-dtype int8 \
--source-file dataset.txt \
--channel-mean-value "128,128,128,128" \
--outdir convert_output \
--target-platform PRODUCT_PID0XA001 \
--quantize-algo kl_asymm_divergence \
--optimize-algo cos-dis \
--kl-threshold-size 4096 \
--divergence-nbins 4096 \
--neg-scale 0.9 \
--disable-per-channel True \
--weights-quantize-algo "mle" \
--weights-optimize-algo "cos-dis" \
--weights-threshold-size 2048 \
--weights-neg-scale 0.999 \
--export-template-code True(default True) \
--system-env 'android'
```

3.5 Parameters

3.5.1 Overview

The parameters listed in 3.4 Model Conversion Procedure are the basic parameters required during model conversion. Generally, the adla file can be generated correctly with the above parameters. This section describes all the parameters available for reference.

3.5.2 adla_convert Parameters

```
usage: adla_convert.py [-h] --model-type MODEL_TYPE --model MODEL
                        [--weights WEIGHTS] [--inputs INPUTS]
                        [--outputs OUTPUTS] [--input-shapes INPUT_SHAPES]
                        [--shape-with-batch SHAPE_WITH_BATCH]
                        [--dtypes DTYPES]
                        [--batch-size BATCH_SIZE] [--iterations ITERATIONS]
                        [--outdir OUTDIR] [--quantize-dtype QUANTIZE_DTYPE]
                        [--source-file SOURCE_FILE]
                        [--channel-mean-value CHANNEL_MEAN_VALUE]
                        [--disable-per-channel DISABLE_PER_CHANNEL]
                        [--inference-input-type INFERENCE_INPUT_TYPE]
                        [--inference-output-type INFERENCE_OUTPUT_TYPE]
                        [--target-platform TARGET_PLATFORM]
                        [--quantize-algo QUANTIZE_ALGO]
                        [--optimize-algo OPTIMIZE_ALGO]
                        [--kl-threshold-size KL_THRESHOLD_SIZE]
                        [--divergence-nbins DIVERGENCE_NBINS]
                        [--percentile PERCENTILE]
                        [--asymm-percentile ASYMM_PERCENTILE]
                        [--moving-alpha MOVING_ALPHA]
                        [--neg-scale NEG_SCALE]
                        [--weights-quantize-algo WEIGHTS_QUANTIZE_ALGO]
                        [--weights-optimize-algo WEIGHTS_OPTIMIZE_ALGO]
                        [--weights-threshold-size WEIGHTS_THRESHOLD_SIZE]
                        [--weights-neg-scale WEIGHTS_NEG_SCALE]
                        [--export-template-code EXPORT_TEMPLATE_CODE]
                        [--system-env SYSTEM_ENV]
```

Argument	Description
--model-dtype(required)	Requires a string value to denote the type of the convert model. Ranges: caffe, onnx, keras, darknet, tensorflow, quantized_tflite, tflite, pytorch, onnx.
--model(required)	Requires a string value to denote the file path of the imported model file, such as xxx.prototxt, xxx.cfg, xxx.onnx
--weights	Requires a string value to denote the file path of the imported weights file, such as xxx.caffemodel, xxx.weights, xxx..params.
--inputs	Multiple points are separated with spaces. For example, "input1 input2"
--input-shapes	Tensor shape sizes of the same input points are separated with commas. For example, '3,224,224', which represents the tensor shape sizes of one input point. Sizes between different input points are separated with hashtags. For example, '3,224,224#3,299,299#12,1', tensor shape sizes of three input points.

--shape-with-batch	Describes whether the --input-shapes contains the highest batch dimension or not. (Optional) None (Default) means --input-size-list should be given without batch. Enclose the bool values in quotation marks, and use # to separate the bool values. Take a graph with two input layer as an example, the three input layers with shape “?x224x224x3”, “11x88”, “10”, input-shapes and shape with batch could be given as: --input-shapes “224,224,3#11,88#10” --shape-with-batch “False#True#True”
--dtypes	Describes the input types (Optional) float32(Default) Enclose the string values in quotation marks, and use # to separate the bool values. Effective value[‘float32’,‘int64’,‘int32’,‘bool’,‘uint8’] Take a graph with three input layer as an example, the three input layers with shape “?x224x224x3”, “11x88”, “10”, input-shapes and shape with batch could be given as: --input-shapes “224,224,3#11,88#10” --shape-with-batch “False#True#True” --dtypes ‘float32#int32#int64’
--outputs	Multiple points are separated with spaces. For example, 'output1 output2'. Note: The current outputs must be set when the model-dtype is tensorflow. When the model-type is onnx/tflite, it does not need to be set by default, but it supports setting the middle layer as an output node.. The Onnx model can directly set the name corresponding to the OUTPUTS of the middle layer. The Tflite model sets the location number corresponding to the middle layer OUTPUTS.
--batch-size	Batch size value. Default value:1
--iterations	Number of iterations to run, integer value. (Optional) 1 (Default)
--outdir	Requires a string value to denote the output path of the generated adla file, default: “xxx_adla_unify” (current folder)
--quantize-dtype	Data type used for quantized data. (Optional) Valid dtype values are: int8: signed 8bit, default per-channel, if per-layer quantization is required, use it with disable-per-channel parameter int16: signed 16bit, default per-channel, if pe-rlayer quantization is required, use it with disable-per-channel parameter uint8: unsigned 8bit
--source-file	Dataset path and filename. The file extension indicates the dataset: xxx.txt, set the image path or npy file path in txt.
--channel-mean-value	Input channels mean value. This parameter should always be given according to running models. Note: This parameter is only valid for int8/int16 quantization and the source-file is image path.
--disable-per-channel	default True

--inference-input-type	After converting to adla, the input data type of the model. Default: same as quantize-dtype For int8 quantize, inference input type only supports float32, int8, uint8. For int16 quantize, inference input type only supports float32, int16. For uint8 quantize, inference input/output type must be all float32, or all uint8.
--inference-output-type	After converting to adla, the output data type of the model. Default: same as quantize-dtype For int8 quantize, inference input type only supports float32, int8, uint8. For int16 quantize, inference input type only supports float32, int16. For uint8 quantize, inference input/output type must be all float32, or all uint8.
--target-platform	Specify the config file to export adla file on the target platform. Valid values as below: PRODUCT_PID0XA001(default) PRODUCT_PID0XA002 PRODUCT_PID0XA003 PRODUCT_PID0XA004
--quantize-algo	1. "kl-<i>asymm</i>_divergence": The kl asymmetric quantization algorithm. If this algorithm is selected, you can set the parameters related to the algorithm: --optimize-algo,--kl-threshold-size ,--divergence-nbins,--neg-scale 2. "kl-<i>symm</i>_divergence": The kl symmetric quantization algorithm. If this algorithm is selected, you can set the parameters related to the algorithm: --optimize-algo,--kl-threshold-size ,--divergence-nbins 3. "percentile": The percentile algorithm,a percentile-based activation clamping quantification method. If this algorithm is selected, you can set the parameters related to the algorithm: --percentile 4. "moving-avg": The moving-avg algorithm. If this algorithm is selected. You can set the parameters related to the algorithm --moving-alpha
--optimize-algo	Quantitative methods to measure the similarity between real and quantified data distributions, only valid for "kl-<i>symm</i>_divergence"/"kl-<i>asymm</i>_divergence" Valid optimize-algo values are: 1. "kl-dis": Default optimize algorithm, kl divergence 2. "js-dis": js divergence, only valid for "kl-<i>asymm</i>_divergence" 3. "cos-dis": cosine similarity,only valid for "kl-<i>asymm</i>_divergence" 4. "mse-dis": mean square error loss,only valid for "kl-<i>symm</i>_divergence"
--kl-threshold-size	Whether the feature uses the kl quantization threshold during kl quantization. If the feature size of the corresponding layer > threshold_size, use it. only valid for "kl-<i>symm</i>_divergence"/"kl-<i>asymm</i>_divergence". 2048(default) The integer must equal 2N where N is a positive integer, recommended values: 2048/4096/8192/12288/16384

--divergence-nbins	only valid for " kl_symm_divergence " / " kl_asymm_divergence " Requires an integer to specify the number of bins in the Kullback-Laiber divergence (KL divergence) histogram. 2048(default) The integer must equal 2N where N is a positive integer, Recommended value: 2048/4096/8192/12288/16384
--percentile	Only valid for " percentile ", 0.9999(default) Suggested value range: 0.999 ~ 0.9999 , step_size 0.0001
--asymm-percentile	Only valid for " percentile ", -1.0(default) Suggested value range: 0.999 ~ 0.9999 , step_size 0.0001 When it is the default value, it is a symmetric percentage quantization algorithm. When set to other values, it is an asymmetric percentage quantization algorithm.
--moving-alpha	only valid for " moving_alpha ", 0.2(default) Suggested value range: 0.2 ~ 0.9, step_size 0.1
--neg-scale	Scale factor for asymmetric quantization. only valid for " kl_asymm_divergence ", 1.0(default) Suggested value range: 0.001/0.01/0.1/1/1.2/1.5
--weights-quantize-algo	The weights quantization algorithm. Valid values are: 1. " min-max ": Default algorithm 2. " mle ": The mle quantization algorithm.
--weights-optimize-algo	Quantitative methods to measure the similarity between real and quantified data distributions, only valid for " mle " Valid optimize-algo values are: 1. " kl-dis ": Default optimize algorithm, kl divergence 2. " js-dis ": js divergence, 3. " cos-dis ": cosine similarity
--weights-threshold-size	The weights shape size threshold for mle quantization of weights. 2048(default) recommended values: 2048/4096/8192/12288/16384
--weights-neg-scale	Scale factor for weights mle quantization. only valid for " mle ", 0.999(default) Suggested value range: 0.99~1.0
--export-template-code	Whether to export demo code default True
--system-env	Specify the single board system environment. Used with --export-template-code ,just to export compiled scripts in different environments. valid values as below: " linux ": default " android "

4. Other Tool

4.1 Adla_info

Adla_info is a tool that can parse generated adla files. Through the adla_tool tool, you can get the platform information, compiler version information, input and output information corresponding to the adla file.

The path where Adla_info tool is integrated in the tool:

adla-toolkit-binary-1.x.x.x/bin/adla_plug_in/tools/adla_info_tool/: contains the following files:

```
adla-toolkit-binary-1.6.10.3/bin/adla_plug_in/tools/adla_info_tool/  
├── adlainfo_release  
├── mobilenet_v1_1.0_224_quant_u8.adla  
├── README_release_EN.txt  
└── README_release.txt
```

Customers can use the adla_tool tool according to the introduction in README_release.txt. Commonly used the commands:

`./adlinfo_release -v mobilenet_v1_1.0_224_quant_u8.adla` can get the version information and platform information corresponding to the adla file.

```
adla file name : mobilenet_v1_1.0_224_quant_u8.adla
*****
Compile_Version : 0.5.3
AmI Hw Version  : 1.0
*****
```

`./adlinfo_release -io mobilenet_v1_1.0_224_quant_u8.adla`, get the model input and output information corresponding to the adla file and the AXI_SRAM size configured in config.

```
adla file name : mobilenet_v1_1.0_224_quant_u8.adla
*****
Inputs                                     : [88 ]
  Input[0]                               : 88
  Dim Count                              : 4
  Size of dim[0]                          : 1
  Size of dim[1]                          : 224
  Size of dim[2]                          : 224
  Size of dim[3]                          : 3
  type                                   : Uint8
  scale                                  : [0.007812 ]
  zero_point                              : [128 ]
  inputs_memory_size                      : 150528
*****
*****
Outputs                                    : [87 ]
  Output[0]                              : 87
  Dim Count                              : 2
  Size of dim[0]                          : 1
  Size of dim[1]                          : 1001
  type                                   : Uint8
  scale                                  : [0.003906 ]
  zero_point                              : [0 ]
  outputs_memory_size                    : 1001
*****
*****
config:
  max_outstanding_outputs : 4
  axi_sram_size            : 262144 bytes
*****
```

4.2 Quantize optimization_tool

The Quantize optimization_tool is an optimal quantization parameter search tool developed based on the Adla_tool tool and adding a series of feature/weights quantification algorithms.

The path where Quantize optimization_tool is integrated in the tool:

`adla-toolkit-binary-1.x.x.x/bin/adla_plug_in/tools/Quantize_optimization_tool/`: contains the following files:

```
Quantize_optimization_tool/
├── Quantize_optimization_tool_guide_CN.txt
├── Quantize_optimization_tool_guide.txt
├── Quantize_optimization_tool.py
└── Quantize_optimization_tool_simple_version.py
```


Customers can use this tool according to the introduction of Model_quantize_guide in this directory.

The current tool exports the quantized IR into a quant tflite model during the model conversion process, and then confirms the quantization effect by running quant tflite. The judgment standard in the current tool is to judge and select the optimal value by counting the cosine similarity and Euclidean distance between the single-frame output results of the original model.

This tool has been used to improve the quantitative accuracy of multiple customers' models.

Tool usage suggestions:

1. If the customer's model has other verification sets, it is recommended to integrate the verification code in the run_tflite interface and search for optimal quantitative parameters based on the customer's own verification results.

2. Search suggestions: 1. Modify self.quantize_algos=['min-max'] in the script. At this time, the quantification effects of different data sets will be traversed. Prioritize an optimal quantization parameter iterations under min-max quantization. 2. Modify the value of self.iteration=200 to the optimal value searched in the first step, and then set self.quantize_algos=['kl_asymm_divergence','kl_symm_divergence','percentile','moving-avg','mle'], conduct the second round of search.

3. For detailed usage suggestions, please refer to the Quantize_optimization_tool_guide.txt document

4.3 NNAPI_Optimize_tool

NNAPI_Optimize_tool is aimed at the problem that the NNAPI runtime does not support some operators, resulting in model splitting and thus affecting the operating efficiency. A set of operators that cause splitting are developed to reconstruct the operators and perform pass optimization in the nnAPI processing flow to improve the model operating efficiency. operator reconstruction tool.

The path where NNAPI_Optimize_tool is integrated in the tool:

adla-toolkit-binary-1.x.x.x/bin/adla_plugin/tools/tflite_refactorer_tool/: contains the following files:

```
tflite_refactorer_tool
├── dped_instance_quant.tflite
├── Readme_CN.txt
├── Readme_EN.txt
└── tflite_refactorer
```

Customers can use this tool according to the introduction in the Readme in this directory. This tool is only needed when customers need to load and run tflite models through the NNAPI process.

4.4 NNIDE_Tool

NNIDE (Neural Network Integrated Development Environment) is a comprehensive neural network development tool based on the nnEngine engine developed by Amlogic.

NNIDE provides Amlogic customers with an efficient and convenient workflow, allowing customers to more quickly evaluate the feasibility and performance of their models on the Amlogic NN platform.

NNIDE_TOOL path integrated in the tool:

adla-toolkit-binary-x.x.x.x/bin/adla_plug_in/tools/ NNIDE_tool , Contains the following files:

```

1. The directory structure is as follows
demo
├── amlnn
│   └── dataset
│       ├── aml_dataset_crop_inception
│       └── input
├── demo_adla_compare_buf.py
├── demo_adla_jupyter.ipynb
├── demo_adla_run_datasets.py
├── demo_adla_run_multi_io.py
├── demo_adla_run_single_io.py
├── model_file
│   ├── adla
│   │   └── original
│   └── NNIDE_package
│       ├── android_32
│       │   └── NNIDE
│       ├── android_64
│       │   └── NNIDE
│       ├── linux_32
│       │   └── NNIDE
│       ├── yocto_64
│       │   └── NNIDE
│       └── requirements.txt
└── Readme.txt

#Compiled file of nnide_tool
#Dataset directory
#Classification model test data set(490 pictures)
#Single frame test data
#Comparison script between the accuracy of the model file saved on the PC and the accuracy of the board end
#Classification model top1/top5 accuracy test script
#Multi-input model inference script
#Single input model inference script
#Test related model files
#Converted adla file
#Original model files, and conversion scripts
#Board end IDE executable file
#Android 32-bit system IDE execution file
#Android 64-bit system IDE execution file
#Linux 64-bit system IDE execution file
#Yocto 64-bit system IDE execution file
#PC environment dependency list

Note:
1. Unzip command: tar -zxvf demo.tar.gz

```

Based on this tool, customers can directly obtain the model running results on the board end in the PC python environment, and directly use python to implement model pre- and post-processing, which can quickly test the accuracy of the model running on the board

Specific reference: NNIDE_1.x.x.docx